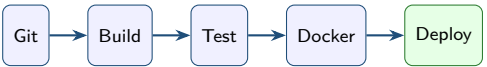
Préparé par

Samir Ouhmmou et Taoufik Mansouri

Jenkins

CI/CD et Jenkinsfile : l'essentiel



24 octobre 2026

Plan

1. Définitions : CI, CD, Jenkins
2. Démarche du lab
3. Installation et configuration
4. Pipelines et Jenkinsfile
5. Directives et steps
6. Labs 1 à 5
7. Webhook et Multibranch
8. Sécurité et bonnes pratiques
9. Commandes de référence
10. Dépannage

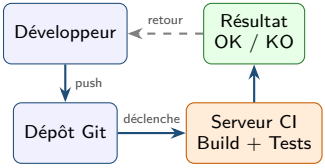
Qu'est-ce que l'intégration continue (CI) ?

Définition

À **chaque modification** du code (commit, push), le projet est **compilé et testé automatiquement** pour détecter les erreurs au plus tôt.

Objectifs

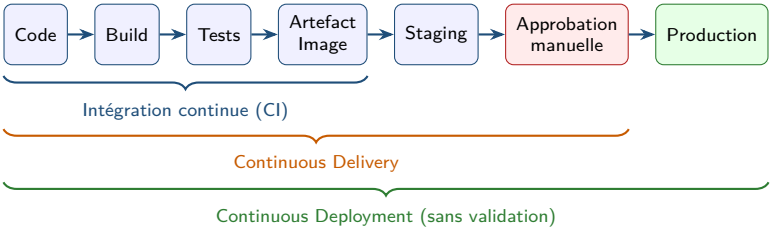
- Détecter les erreurs le plus tôt possible
- Éviter les conflits d'intégration tardifs
- Garder la branche principale saine
- Retour rapide aux développeurs



Livraison continue vs déploiement continu (CD)

Continuous Delivery L'application validée est prête pour la production (artefact, image Docker). Mise en production manuelle.

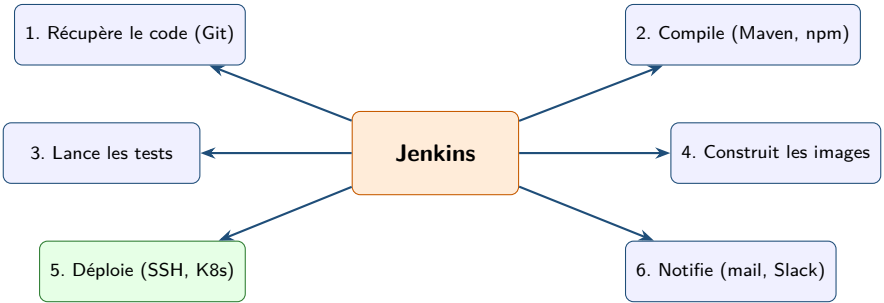
Continuous Deployment Mise en production entièrement automatique si tous les tests passent.



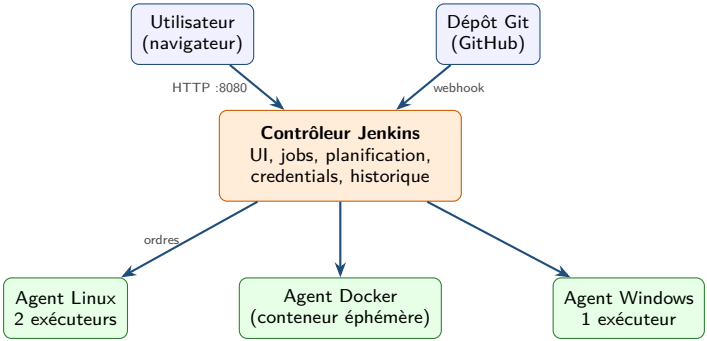
Qu'est-ce que Jenkins et que fait-il ?

Définition

Jenkins est un serveur d'automatisation **open source** (Java) qui exécute les tâches du cycle de vie d'un logiciel. Ses pipelines sont décrits dans un fichier : le Jenkinsfile. Plus de 1800 plugins.



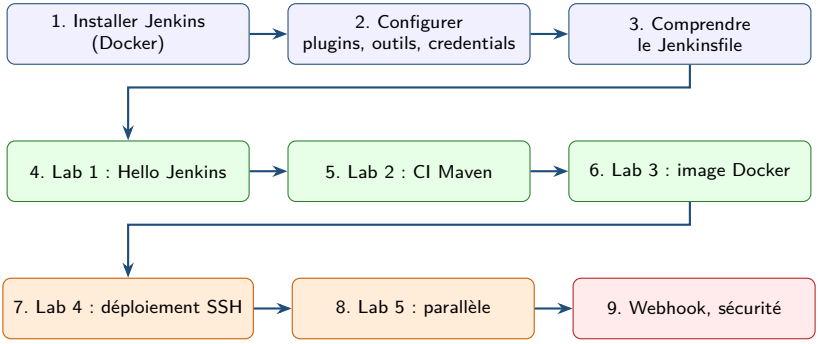
Architecture : contrôleur et agents



Vocabulaire indispensable

Job / Build	Tâche configurée dans Jenkins / une exécution numérotée (#1, #2…).
Pipeline	Suite d'étapes automatisées décrite sous forme de code.
Jenkinsfile	Fichier (Groovy) versionné avec le code, qui décrit le pipeline.
Stage / Step	Grande phase du pipeline / action élémentaire (`sh`, `echo`…).
Agent / Executor	Machine ou conteneur qui exécute le build / emplacement de build.
Workspace	Dossier de travail sur l'agent (code + fichiers générés).
Artefact	Fichier produit par un build (JAR, ZIP, rapport).
Credential	Secret stocké chiffré (mot de passe, token, clé SSH).
Webhook / Trigger	Appel HTTP de GitHub / mécanisme de déclenchement.

Notre démarche : feuille de route



Installer Jenkins avec Docker

```
docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --restart unless-stopped \
  jenkins/jenkins:lts-jdk17

docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

`-d --name jenkins`	Conteneur en arrière-plan, nommé `jenkins`.
`-p 8080:8080 / -p 50000:50000`	Interface web / connexion des agents distants.
`-v jenkins_home:...`	Volume persistant : jobs et plugins conservés.
`-v ...docker.sock`	Accès au Docker de l'hôte (lab uniquement).
`--restart unless-stopped`	Redémarrage automatique.
`docker exec ... cat ...`	Affiche le mot de passe administrateur initial.

Puis `http://localhost:8080` : **Install suggested plugins**, créer l'administrateur.

Configuration : plugins, outils, credentials

Plugins (Manage Jenkins → Plugins)

- Pipeline, Git, GitHub Integration
- Docker Pipeline, Credentials Binding
- SSH Agent, JUnit, Timestamper
- Workspace Cleanup

Tools (Manage Jenkins → Tools)

- JDK : nom `JDK-17`
- Maven : nom `Maven-3.9`

Attention

Le nom doit être **identique** à celui du Jenkinsfile (`tools { maven 'Maven-3.9' }`).

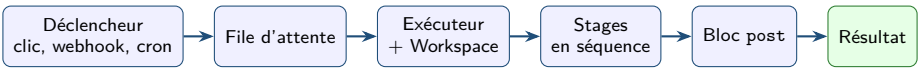
Credentials (Manage Jenkins → Credentials)

ID	Type
`dockerhub-creds`	Username + password / token
`github-creds`	Username + Personal Access Token
`deploy-ssh-key`	SSH Username + clé privée

Le champ **ID** est la valeur référencée dans le Jenkinsfile.

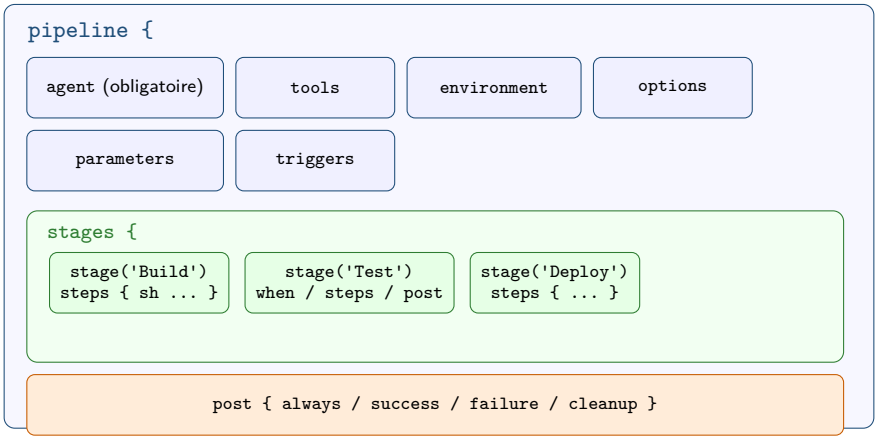
Pipelines : types et cycle d'un build

	Déclaratif (recommandé)	Scripté
Racine	`pipeline { ... }`	`node { ... }`
Structure	Sections prédéfinies, validée avant exécution	Groovy libre, erreurs à l'exécution
Erreurs	Bloc `post`	`try/catch/finally`



Résultats : `SUCCESS`, `UNSTABLE` (tests en échec), `FAILURE`, `ABORTED`. Job : **New Item** → **Pipeline** → **Pipeline script from SCM** (Git, branche `*/main`, Script Path `Jenkinsfile`).

Structure d'un Jenkinsfile déclaratif



Jenkinsfile de référence (1/2) : en-tête

```
pipeline {
    agent any
    tools { maven 'Maven-3.9'; jdk 'JDK-17' }
    environment {
        APP_NAME = 'demo-app'
        VERSION  = "1.0.${BUILD_NUMBER}"
    }
    options {
        timestamps()
        timeout(time: 15, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
        disableConcurrentBuilds()
    }
    parameters {
        booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Tests ?')
    }
    triggers { pollSCM('H/5 * * * *') }
```

Jenkinsfile de référence (2/2) : stages et post

```
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build') {
            steps { sh 'mvn -B clean compile' }
        }
        stage('Test') {
            when { expression { params.RUN_TESTS } }
            steps { sh 'mvn -B test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('Package') {
            steps {
                sh 'mvn -B package -DskipTests'
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
        }
    }
    post {
        success { echo "Build ${VERSION} reussi" }
        failure { echo 'Build en echec' }
        always  { cleanWs() }
    }
}
```

Directives : agent, tools, environment, options

```
agent any              // tout agent
agent none             // un agent par stage
agent { label 'linux' }
agent {
  docker {
    image 'maven:3.9-eclipse-temurin-17'
    args  '-v $HOME/.m2:/root/.m2'
  }
}
environment {
  VERSION = "1.0.${BUILD_NUMBER}"
  CREDS   = credentials('dockerhub-creds')
}
```

- `agent` : obligatoire, où s'exécute le pipeline. `docker` = conteneur éphémère, rien à installer.
- `tools` : ajoute Maven/JDK au `PATH` (noms de Manage Jenkins → Tools).
- `environment` : variables. Guillemets **simples** = littéral, **doubles** = interpolation `${VAR}`.
- `credentials('id')` crée `CREDS`, `CREDS_USR`, `CREDS_PSW` (masqués `****`).
- `options` : `timestamps()`, `timeout(...)`, `buildDiscarder(...)`, `disableConcurrentBuilds()`.

parameters, triggers, when

```
parameters {
  choice(name: 'ENV', choices: ['dev','staging','prod'],
      description: 'Cible')
  booleanParam(name: 'RUN_TESTS', defaultValue: true,
      description: 'Tests')
}
triggers {
  cron('H 2 * * *')         // chaque nuit
  pollSCM('H/5 * * * *')    // Git toutes les ~5 min
  // githubPush()           // webhook
}
stage('Deploy') {
  when { allOf {
      branch 'main'
      expression { params.ENV == 'prod' }
  } }
  steps { echo 'Deploiement' }
}
```

- `params.ENV` ou `$ENV` dans `sh`.
- Bouton : **Build with Parameters**.
- Cron : `MINUTE HEURE JOUR MOIS JOURSEM`, `H` répartit la charge.
- `when` : `branch`, `expression`, `allOf`, `anyOf`, `not`. Condition fausse : stage sauté.

`H/15 * * * *`	15 min
`H 2 * * *`	2h00
`H 8 * * 1-5`	lun-ven 8h

Les steps les plus utilisés

`echo 'message'`	Affiche un message dans le log.
`sh 'commande'`	Commande shell ; échoue si code retour $\neq 0$ (`/bin/sh -xe`).
`checkout scm`	Récupère le code du dépôt du job.
`withCredentials([...]) { }`	Injecte temporairement des secrets.
`sshagent(['id']) { }`	Charge une clé SSH le temps du bloc.
`junit 'chemin/*.xml'`	Publie les résultats de tests.
`archiveArtifacts artifacts: '...'`	Conserve des fichiers produits.
`stash` / `unstash`	Transmet des fichiers entre stages ou agents.
`timeout` / `retry(3)` / `sleep`	Limite, relance, pause.
`input message: '...'`	Attend une validation manuelle.
`script { ... }`	Bloc Groovy libre dans un pipeline déclaratif.
`cleanWs()`	Vide le workspace.

`sh(script: '...', returnStdout: true).trim()` renvoie la sortie ; `returnStatus: true` renvoie le code de retour.

Bloc post et variables prédéfinies

`always`	Toujours.
`success`	Si le build réussit.
`failure`	Si le build échoue.
`unstable`	Tests en échec.
`changed`	Résultat différent du précédent.
`fixed`	Réussi après un échec.
`cleanup`	Toujours, en dernier.

`BUILD_NUMBER`	N° du build.
`BUILD_URL`	URL du build.
`JOB_NAME`	Nom du job.
`WORKSPACE`	Chemin du workspace.
`NODE_NAME`	Agent utilisé.
`GIT_COMMIT`	Hash du commit.
`BRANCH_NAME`	Branche (multibranch).

Lab 1 : « Hello Jenkins »

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello Jenkins !'
                sh 'whoami'
                sh 'pwd'
                sh 'date'
            }
        }
        stage('Environnement') {
            steps {
                echo "Job : ${env.JOB_NAME} #${env.BUILD_NUMBER
     }"
                sh 'printenv | sort | head -20'
            }
        }
    }
}
```

Étapes

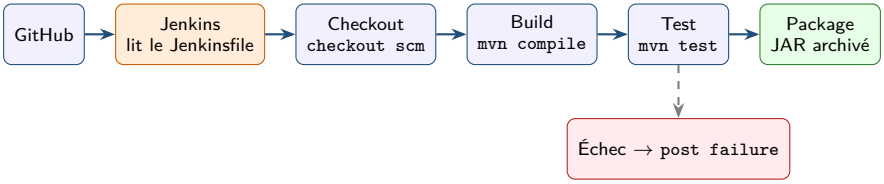
1. New Item → `lab1-hello`, type Pipeline
2. Coller le script, Save
3. **Build Now**
4. Build #1 → **Console Output**

Commandes

- `whoami` : utilisateur `jenkins`
- `pwd` : workspace
- `printenv` : variables du build

Lab 2 : CI d'un projet Maven

```
mvn archetype:generate -DgroupId=com.example -DartifactId=demo-app \
    -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd demo-app && git init && git add . && git commit -m "Initial commit"
mvn -B clean package        # doit reussir en local avant de pousser
git remote add origin https://github.com/VOTRE_COMPTE/demo-app.git
git branch -M main && git push -u origin main
```



`-B` : mode batch (logs lisibles) ; `clean` : supprime `target/` ; `test` : Surefire, rapports dans `target/surefire-reports/` ; `package` `-DskipTests` : JAR sans retester. Dans le `pom.xml` : Java 17 (`maven.compiler.source/target`) et `mainClass` dans `maven-jar-plugin`. Utiliser le Jenkinsfile de référence.

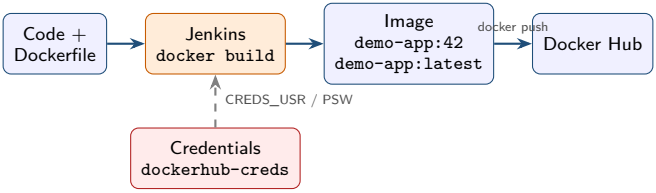
Lab 3 : image Docker, Dockerfile et stages

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

`FROM` image JRE 17 légère ; `COPY` copie le JAR ; `EXPOSE` documente le port ; `ENTRYPOINT` commande de démarrage.

```
environment {
  IMAGE        = 'votrecompte/demo-app'
  TAG          = "${BUILD_NUMBER}"
  DOCKER_CREDS = credentials('dockerhub-creds')
}
stage('Docker Build') {
  steps { sh 'docker build -t $IMAGE:$TAG -t $IMAGE:latest .' }
}
stage('Docker Push') {
  when { branch 'main' }
  steps {
    sh 'echo "$DOCKER_CREDS_PSW" | docker login -u "
     $DOCKER_CREDS_USR" --password-stdin'
    sh 'docker push $IMAGE:$TAG'
    sh 'docker push $IMAGE:latest'
  }
  post { always { sh 'docker logout' } }
}
```

Lab 3 : explications et vérification



- `-t $IMAGE:$TAG -t $IMAGE:latest .` : deux tags, le `.` final est le contexte de build.
- Guillemets simples : le shell remplace les variables, le secret n'est pas interpolé par Groovy.
- `--password-stdin` : le mot de passe n'apparaît pas dans ps.
- `when { branch 'main' }` : job Multibranch requis (sinon tester `env.GIT_BRANCH`).

```
docker run -d --name test-app -p 9090:8080 votrecompte/demo-app:latest
curl http://localhost:9090      # -> Hello from Jenkins CI/CD!
docker logs test-app && docker rm -f test-app
```

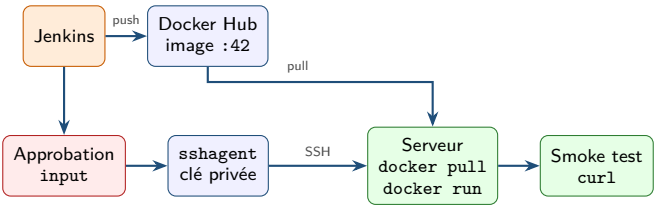
Lab 4 : déploiement SSH avec approbation

```
stage('Approbation') {
  when { branch 'main' }
  steps {
    timeout(time: 10, unit: 'MINUTES') {
      input message: 'Deployer en production ?'
    }
  }
}
stage('Deploy') {
  when { branch 'main' }
  steps {
    sshagent(credentials: ['deploy-ssh-key']) {
      sh '''
        ssh -o StrictHostKeyChecking=no deploy@192.168.1.50 "
          docker pull $IMAGE:$TAG &&
          docker rm -f demo-app || true &&
          docker run -d --name demo-app -p 80:8080 $IMAGE:$TAG"
      '''
    }
  }
}
```

```
ssh-keygen -t ed25519 -f jenkins_deploy -N ""
ssh-copy-id -i jenkins_deploy.pub deploy@192
      .168.1.50
```

- `ssh-keygen` : paire de clés ; `ssh-copy-id` : installe la clé publique sur le serveur.
- Clé *privée* → credential `deploy-ssh-key`.
- `input` met le pipeline en pause (avec `timeout`).
- `sshagent` charge la clé le temps du bloc.
- `|| true` : pas d'échec si le conteneur n'existe pas.
- `StrictHostKeyChecking=no` : lab uniquement.

Lab 4 : schéma du déploiement et smoke test

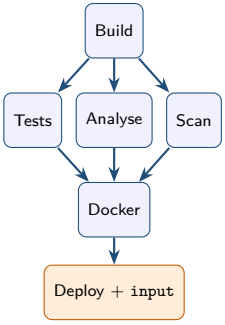



```
stage('Smoke test') {
  steps {
    retry(5) {
      sleep time: 3, unit: 'SECONDS'
      sh 'curl -fsS http://192.168.1.50/ | grep "Hello"'
    }
  }
}
```

`curl -f` échoue si HTTP ≥ 400, `-sS` silencieux sauf erreur ; `retry(5)` laisse l'application démarrer.

Lab 5 : stages parallèles et pipeline final

```
parameters {
  choice(name: 'ENV', choices: ['dev','staging','prod'])
  booleanParam(name: 'SKIP_TESTS', defaultValue: false)
}
stage('Qualite') {
  when { expression { !params.SKIP_TESTS } }
  parallel {
    stage('Tests unitaires') {
      steps { sh 'mvn -B test' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Analyse') { steps { sh 'mvn -B verify -DskipTests' } }
    stage('Scan Dockerfile') { steps { sh 'grep -n FROM
     Dockerfile' } }
  }
}
stage('Deploy') {
  when { expression { params.ENV != 'dev' } }
  steps { input message: "Deployer ${VERSION} ?" }
}
```



`parallel` : sous-stages simultanés ; `failFast true` arrête les autres si l'un échoue.

Webhook GitHub et pipeline Multibranch



Webhook

1. GitHub : Settings → Webhooks → Add
2. URL : `http://JENKINS:8080/github-webhook/`
3. Content type `application/json`, événement *push*
4. Jenkinsfile : `triggers { githubPush() }`

Poste local : `ngrok http 8080`.

Multibranch

- New Item → **Multibranch Pipeline**
- Un job créé automatiquement par branche et par pull request
- `branch 'main'`, `BRANCH_NAME`, `changeRequest()` fonctionnent
- Déploiement uniquement depuis `main`

Sécurité et bonnes pratiques

```groovy
// MAUVAIS : interpolation Groovy d'un secret (visible dans les logs)
sh "curl -H 'Authorization: Bearer ${TOKEN}' https://api.exemple.com"
// BON : guillemets simples, le shell lit la variable
withCredentials([string(credentialsId: 'api-token', variable: 'TOKEN')]) {
    sh 'curl -H "Authorization: Bearer $TOKEN" https://api.exemple.com'
}
```

- Jenkinsfile dans Git (*Pipeline as Code*)
- Secrets uniquement via *Credentials*
- Contrôleur léger : builds sur des agents
- Authentification, droits minimaux, HTTPS
- Toujours `timeout` et `buildDiscarder`
- `cleanWs()` et `docker logout` en fin de build
- Jenkins et plugins à jour (LTS), sauvegarde de `JENKINS_HOME`
- Approbation manuelle avant la production

Commandes de référence

Service / Docker	
`systemctl status jenkins`	État du service.
`journalctl -u jenkins -f`	Logs en direct.
`docker ps`	Conteneurs actifs.
`docker logs -f jenkins`	Logs du conteneur.
`docker exec -it jenkins bash`	Shell dans le conteneur.
`docker build -t n:t .`	Construit une image.
`docker system prune -af`	Nettoie le disque.

Git / Maven / Jenkins	
`git push origin main`	Envoie sur `main`.
`git rev-parse --short HEAD`	Hash court.
`mvn -B clean package`	Build en mode CI.
`mvn -DskipTests`	Saute les tests.
`jenkins-cli.jar list-jobs`	Liste les jobs.
`jenkins-cli.jar build JOB -s -v`	Lance un build.
`curl -X POST -u u:TOKEN .../job/X/build`	API REST.

Valider un Jenkinsfile sans l'exécuter : `curl -X POST -u u:TOKEN -F "jenkinsfile=<Jenkinsfile" http://localhost:8080/pipeline-model-converter/validate`

Dépannage : erreurs courantes

Symptôme	Cause	Solution
`docker: command not found`	Client Docker absent.	Image personnalisée ou agent Docker.
`permission denied ... docker.sock`	`jenkins` hors du groupe `docker`.	`usermod -aG docker jenkins` ; `--group-add`.
`mvn: command not found`	Outil absent du `PATH`.	Déclarer `tools { maven '...' }`.
`Tool type "maven" does not have an install of ...`	Nom différent de celui des Tools.	Aligner exactement les noms.
`BRANCH_NAME` nul	Job non multibranch.	Multibranch ou `GIT_BRANCH`.
`unexpected token`	Syntaxe Groovy.	Endpoint `validate`.
`Host key verification failed`	Empreinte SSH inconnue.	Ajouter à `known_hosts`.
`Authentication failed` (Git)	Token expiré.	Recréer le Personal Access Token.
Webhook sans effet	URL ou accès réseau.	*Recent Deliveries* ; URL en `/github-webhook/`.
`No space left on device`	Disque saturé.	`cleanWs()`, `docker system prune -af`.

Méthode : lire la première erreur dans Console Output ; **Replay** pour rejouer sans commit ; reproduire avec `docker exec -it -u jenkins jenkins bash`.

Récapitulatif et conclusion

1. Jenkinsfile dans Git, à la racine
2. Un stage = une responsabilité
3. `timeout` et `buildDiscarder` toujours
4. Secrets via Credentials, guillemets simples
5. Publier les tests même en échec (`post always`)
6. Nettoyer : `cleanWs()`, `docker logout`
7. Agents Docker pour des builds reproductibles
8. Approbation avant la production

Merci !

Des questions ?

Samir Ouhmmou & Taoufik Mansouri

